The Journal of Open Source Software

URSA: The Universal Research and Scientific Agent

Russell Bent [1], **Tristan Brennan** [2], **Nathan DeBardeleben** [1], **Warren D. Graham** [1], **Pascal Grosset** [1], **Michael Grosskopf** [1], **Adela Habib** [1], **Earl Lawrence** [1], **Arthur Lui** [1], **Isaac Michaud** [1], **Avanish Mishra** [1], **Jamal Mohd-Yusof** [1], **Harsha Nagarajan** [1], **Sachin Shivakumar** [1], **Rahul Somasundaram** [1], **Jemma Stachelek** [1], **Joan Vendrell Gallart** [1], **Alexius Wadell** [1], **and Golo A. Wimmer** [1]

1 Los Alamos National Laboratory, Los Alamos, New Mexico, United States **2** Stanford University, Stanford, California, United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Chris Vernon

Reviewers:

- @rahuldevikar
- @vbcherepanov

Submitted: 04 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Agentic workflows, i.e. software systems that use large language models (LLMs) (Zhao et al., 2026), are increasingly being used to automate complex tasks such as software engineering and information retrieval. These developments have created new opportunities for AI systems that assist scientific research. Here, we present URSA (Universal Research and Scientific Agent), an open-source framework for building modular and extensible AI agents specifically for scientific workflows. URSA enables researchers to compose domain-specific agents by integrating LLMs with the appropriate scientific computational tools, supporting both general-purpose scientific reasoning and specialized research applications.

Statement of need

Scientific research increasingly requires AI systems that can integrate reasoning with external software, simulation codes, computational resources, and specialized literature (Boiko et al., 2023). While existing agentic frameworks provide general-purpose planning and tool use, researchers often need specialized agents tailored to particular research questions.

The URSA framework addresses this need through a collection of reusable core agents that implement common research capabilities such as hypothesis generation, planning, and execution. Furthermore, building on these core agents, URSA includes several specialized agents that demonstrate how the framework can be extended to research applications by integrating scientific computational tools and domain knowledge. Researchers can interact with the framework through a command-line interface, web dashboard, Model Context Protocol (MCP), or Python API, while execution environments enable multi-agent collaborations.

State of the field

Coding agents such as Claude Code (Anthropic, 2026) and Codex (OpenAI, 2026) assist with software development, including code generation and debugging. Scientific assistants such as AI Scientist (Lu et al., 2024), Co-Scientist (Gottweis et al., 2026), SciAgents (Ghafarollahi & Buehler, 2025), Agent Laboratory (Schmidgall et al., 2025), and ChemCrow (Bran et al., 2024) demonstrate the application of LLMs to research. Frameworks for constructing agents provide complementary capabilities: LangGraph (LangChain Inc., 2026) supports stateful graph workflows and checkpointing, AutoGen (Wu et al., 2024) provides abstractions for multi-agent conversations and teams, CrewAI (CrewAI, 2026) organizes agents into crews and workflows,

and smolagents (Roucher et al., 2026) supports agents that act through generated code or tool calls. These frameworks provide general mechanisms on which scientific applications can be built. However, in building URSA, we noted that explicit control over agent execution and persistent state for scientific tasks was desirable. We therefore chose LangGraph's stateful graph abstraction over conversation-, crew-, and code-based abstractions emphasized by AutoGen, CrewAI, and smolagents, respectively.

URSA builds on LangGraph by packaging reusable research agents, agents for running scientific software and simulation codes, and interfaces for configuring and running scientific workflows. Its contributions include reusable agents for hypothesis generation, planning, simulation, and optimization, together with collaboration environments such as Agent Teams and Agent Symposia. Maintaining these components in a separate package allows domain-specific dependencies, examples, and user interfaces to evolve together while reusing LangGraph's orchestration infrastructure. Researchers can thus extend these implementations for their own scientific tools and tasks.

Software design

URSA uses LangGraph (LangChain Inc., 2026) to express agent workflows through explicit states, nodes, and transitions, and to support checkpointing. This makes stages such as planning, review, execution, and error recovery independently configurable. Reusing this infrastructure reduces the orchestration code maintained within URSA and allows it to take advantage of LangGraph's state model. However, this dependence on LangGraph's stateful graph-based workflows does require URSA to express state transitions and control flows within LangGraph's abstractions, potentially making it more difficult to integrate agents that do not naturally fit this model. URSA provides support for multiple LLM endpoints through LangChain's chat-model interface. Examples of models and inference providers that we have used with URSA include GPT models from GPT-4 onward and o-series models through the OpenAI API, Claude models from Sonnet 3.5 onward through Amazon Bedrock, and the open-weight model gpt-oss-120b through Ollama.

Agent Framework

URSA provides reusable core agents and domain-specific agents that combine them with scientific tools. The agents listed below illustrate these capabilities and are not an exhaustive list. Each agent has a configurable workspace for working files and command execution. Named agents maintain their persistent states in separate directories organized by group and agent name. Thread identifiers distinguish checkpointed conversations and checkpoints preserve the agent's state.

The `run_command` tool normally asks an LLM to assess a proposed shell command before executing it. On Linux and macOS, shell commands are sandboxed by default to restrict filesystem writes, while reads and network access remain enabled. The environment variable `URSA_SAFETY_LEVEL` is used to assess safety: `default` assesses commands with explicit trust assumptions for configured executables, agent-created files, and LaTeX operations; `low` focuses on destructive commands and high-concern data extraction; `trusted` supplies a generic trust statement without the command; `none` bypasses assessment while retaining sandboxing; and `yolo` disables both. LLM screening adds inference cost and can incorrectly accept or reject commands.

Core Agents

Examples of core agents include:

- Planning Agent: Generates, reviews, and formalizes research plans as structured tasks.

DRAFT

- Execution Agent: Carries out tasks using tools for code execution, file operations, and shell commands, with additional tools available through the Model Context Protocol (MCP).

Domain-Specific Agents

Examples of domain-specific agents include:

- Simulation Agent: Coordinates two Chat Agents for documentation acquisition and simulation setup, execution, and debugging.
- Optimization Agent: Formulates optimization problems, selects solvers, generates code, and checks results.
- LAMMPS Agent: Selects interatomic potentials, generates simulation inputs, runs the Large-scale Atomic/Molecular Massively Parallel Simulator (LAMMPS) (Thompson et al., 2022), and supports error recovery (Somasundaram et al., 2026).

User Interfaces

Users can interact with URSA through:

- The Command Line Interface (CLI): The CLI starts with:

  ```
  ursa --config config.yaml
  ```

  The configuration selects models and runtime options. Within the REPL, agent names prefixed with # route requests:

  ```
  ursa> #plan Write a workflow for computing elastic constants
        using a molecular dynamics simulation.
  ursa> #execute Execute the plan.
  ```

- A Web Dashboard: The web dashboard provides a graphical interface and can be launched with `ursa-dashboard` after installing the optional dashboard dependencies.
- Python Interface: Python supports embedding agents in scripts and notebooks:

  ```python
  from langchain.chat_models import init_chat_model
  from langchain_core.messages import HumanMessage
  from ursa.agents import ExecutionAgent

  llm = init_chat_model(model="openai:gpt-5.4")
  agent = ExecutionAgent(
      llm=llm,
      workspace="./ursa-script-workspace",
  )

  result = agent.invoke({
      "messages": [
          HumanMessage(
              content=(
                  "Write and run a Python script that prints "
                  "the first 10 prime numbers."
              )
          )
      ],
  })
  print(result["messages"][-1].content)
  ```

DRAFT

Agent Execution Environments

URSA provides coordination environments for delegation and comparison of alternative agent processes. Examples of these environments include Agent Teams and Agent Symposia.

- Agent Teams: use a principal investigator agent to delegate tasks and combine results. The Execution Agent serves as the default coordinator, although users can substitute other agents equipped with delegation tools.
- Agent Symposia: let participants independently propose solutions, critique one another, and refine their responses before synthesis. Participants can bring different tools, models, and assumptions to the discussion.

Research impact statement

URSA has been applied by the authors to scientific workflows, including inertial confinement fusion design and atomistic simulation. Developed at Los Alamos National Laboratory, it is distributed as open-source software to support use and contributions by the broader scientific community.

As one example, Figure 1 shows the use of URSA in the design of inertial confinement fusion (ICF) capsules. In this workflow, URSA's hypothesizer and execution agents explored candidate designs to optimize neutron yield. More details of this application can be found in (Grosskopf et al., 2025).

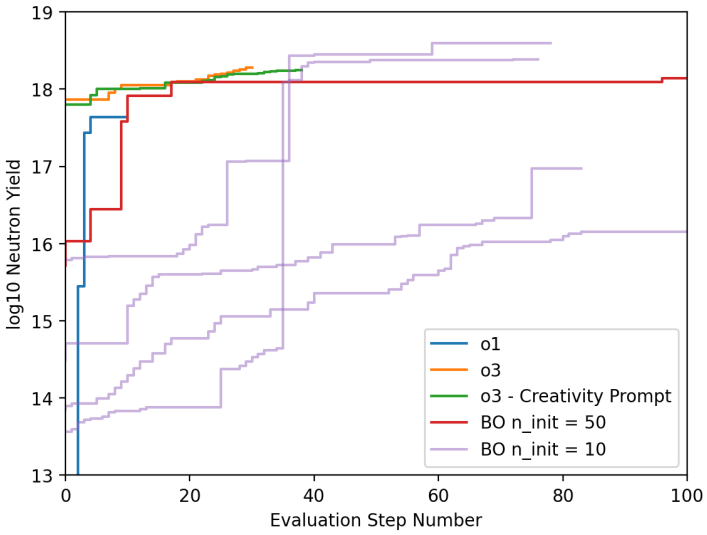


Figure 1: Comparison of URSA to Bayesian optimization (BO) for direct-drive ICF design, showing the running maximum neutron yield. URSA received a published double-shell ICF paper as literature context and a performance-target hint of $\log_{10}$ yield above 17; BO received neither. The initial Latin-hypercube evaluations for BO are omitted from the plot, and each URSA configuration was run once. Figure reproduced from Grosskopf et al. (2025); the complete ICF workflow is not included in this repository.

As a second example, Figure 2 shows atomistic calculations performed by URSA's LAMMPS Agent (Somasundaram et al., 2026) for a high-entropy alloy (George et al., 2019). Such calculations are commonly employed in computational materials science to predict material properties and guide the design of novel materials.

The simulation results shown in this figure were generated using the example script `stiffness_tensor_of_hea.py`.

DRAFT

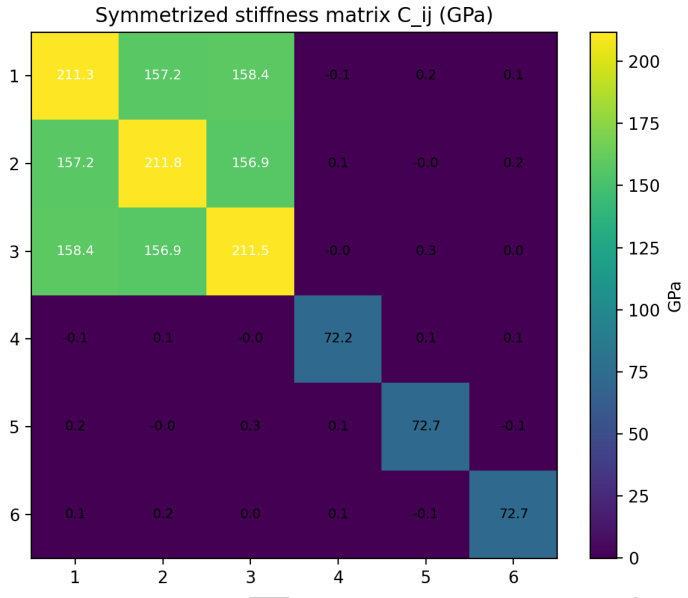


Figure 2: The stiffness tensor, i.e., the elastic constants calculated for the high entropy alloy Co-Cr-Fe-Mn-Ni. The atomistic calculation was performed by the LAMMPS agent. The results use a single random atomic configuration, without configuration averaging.

AI usage disclosure

This work is on AI-driven agentic workflows, so LLMs are invoked at multiple instances within URSA. For example, LLMs were involved in the generation of Figure 1 and Figure 2. During the development of URSA, generative AI tools such as Codex and URSA itself were used to assist with implementing specific portions of the source code, including its documentation. While all major design and architectural decisions were made by the authors, generative AI tools were used for tasks such as prototyping new features, debugging code, and writing tests. For the writing of the manuscript, including its updates during the review process, generative AI tools were used for grammar and spell checks, cutting down the length of the manuscript, and for checking consistency of the manuscript with reviewer comments. All LLM-generated text, code, and documentation were critically reviewed by the authors for accuracy.

Acknowledgements

This work was supported by the Laboratory Directed Research and Development program of Los Alamos National Laboratory under project number 20250638DI. This research used resources provided by the Los Alamos National Laboratory Institutional Computing Program, which is supported by the U.S. Department of Energy National Nuclear Security Administration under Contract No. 89233218CNA000001. This document has been approved for unlimited release, and was assigned LA-UR-26-27324.

References

Anthropic. (2026). *Claude Code (version 2.1.312)*. GitHub. https://github.com/anthropics/claude-code/releases/tag/v2.1.312

Boiko, D. A., MacKnight, R., Kline, B., & Gomes, G. (2023). Autonomous chemical research with large language models. *Nature*, *624*(7992), 570–578. https://doi.org/10.1038/s41586-023-06792-0

DRAFT

Bran, A. M., Cox, S., Schilter, O., Baldassari, C., White, A. D., & Schwaller, P. (2024). Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, *6*(5), 525–535. https://doi.org/10.1038/s42256-024-00832-8

CrewAI. (2026). *CrewAI (version 1.15.32)*. GitHub. https://github.com/crewAIInc/crewAI/releases/tag/1.15.32

George, E. P., Raabe, D., & Ritchie, R. O. (2019). High-entropy alloys. *Nature Reviews Materials*, *4*(8), 515–534. https://doi.org/10.1038/s41578-019-0121-4

Ghafarollahi, A., & Buehler, M. J. (2025). SciAgents: Automating scientific discovery through bioinspired multi-agent intelligent graph reasoning. *Advanced Materials*, *37*(22), 2413523. https://doi.org/10.1002/adma.202413523

Gottweis, J., Weng, W.-H., Daryin, A., Tu, T., Sirkovic, P., Myaskovsky, A., Glowaty, G., Weissenberger, F., Orlandi, A., Popovici, D., & others. (2026). Accelerating scientific discovery with co-scientist. *Nature*, *655*, 487–496. https://doi.org/10.1038/s41586-026-10644-y

Grosskopf, M., Debardeleben, N., Bent, R., Somasundaram, R., Michaud, I., Lui, A., Wadell, A., Graham, W. D., Wimmer, G. A., Shivakumar, S., & others. (2025). Ursa: The universal research and scientific agent. *arXiv Preprint arXiv:2506.22653*. https://doi.org/10.48550/arXiv.2506.22653

LangChain Inc. (2026). *LangGraph (version 1.2.18)*. GitHub. https://github.com/langchain-ai/langgraph/releases/tag/1.2.18

Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., & Ha, D. (2024). The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv Preprint arXiv:2408.06292*. https://doi.org/10.48550/arXiv.2408.06292

OpenAI. (2026). *Codex (version 0.165.0)*. GitHub. https://github.com/openai/codex/releases/tag/rust-v0.165.0

Roucher, A., Villanova del Moral, A., Wolf, T., Werra, L. von, & Kaunismäki, E. (2026). *smolagents: A smol library to build great agentic systems (version 1.26.0)*. GitHub. https://github.com/huggingface/smolagents/releases/tag/v1.26.0

Schmidgall, S., Su, Y., Wang, Z., Sun, X., Wu, J., Yu, X., Liu, J., Moor, M., Liu, Z., & Barsoum, E. (2025). Agent laboratory: Using LLM agents as research assistants. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 5977–6043. https://doi.org/10.18653/v1/2025.findings-emnlp.320

Somasundaram, R., Habib, A., Dang, K., Shivakumar, S., Hill, R. G., Wimmer, G., Mishra, A., Pachalieva, A., Lui, A., Viswanathan, H., Grosskopf, M., Fensin, S., Bent, R., DeBardeleben, N., & Lawrence, E. (2026). An Agentic Orchestration of Atomistic Simulations. *arXiv e-Prints*, arXiv:2607.22596. https://doi.org/10.48550/arXiv.2607.22596

Thompson, A. P., Aktulga, H. M., Berger, R., Bolintineanu, D. S., Brown, W. M., Crozier, P. S., In't Veld, P. J., Kohlmeyer, A., Moore, S. G., Nguyen, T. D., & others. (2022). LAMMPS-a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, *271*, 108171. https://doi.org/10.1016/j.cpc.2021.108171

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversations. *First Conference on Language Modeling*. https://openreview.net/forum?id=BAakY1hNKS

Zhao, W. X., Zhou, K., Li, J., Tang, T., Dong, Z., Hou, Y., Zhang, B., Min, Y., Zhang, J., Liu, P., & others. (2026). A survey of large language models. *Frontiers of Computer Science*, *20*(12), 2012627. https://doi.org/10.1007/s11704-026-60308-3

DRAFT